

DESARROLLO DE APLICACIONES WEB
Examen Parcial

AI Models Explorer

Aplicación web para explorar modelos de inteligencia artificial

Stack tecnológico:

React • Vite • Axios • TailwindCSS • OpenRouter API

Alumna: Alyson Pamela Alvarado Jovel

Tutora: Victoria Castro

Carnet : AJ24002

Enlace de video explicativo :

https://drive.google.com/file/d/1WI83Qfzp8J6D1mSp5mrw_nTwpUZ8O1O2/view?usp=sharing

1. Introducción del Proyecto

En el marco del curso de Desarrollo de Aplicaciones Web, se presenta el proyecto denominado AI Models Explorer, una aplicación web de pagina unica (SPA) desarrollada con React y Vite que consume la API publica de OpenRouter para visualizar, explorar y comparar los modelos de inteligencia artificial disponibles en dicha plataforma.

El proyecto nace de la necesidad de contar con una interfaz unificada, intuitiva y moderna que permita a desarrolladores, investigadores y entusiastas de la tecnología conocer las características técnicas de los principales modelos de IA disponibles en el mercado actual. OpenRouter es una plataforma que agrega y expone, a través de una API REST estandarizada, mas de trescientos modelos de distintos proveedores como OpenAI, Anthropic, Google, Meta, Mistral y Microsoft, entre otros.

La aplicación permite realizar búsquedas en tiempo real por nombre o proveedor, visualizar información clave de cada modelo como su ventana de contexto, precio por millón de tokens de entrada y salida, y su identificador técnico único. Además, el usuario puede seleccionar un modelo específico para resaltarlo visualmente y acceder a su descripción completa.

Desde el punto de vista técnico, el proyecto implementa de manera deliberada y documentada todos los requerimientos exigidos en la rubrica del parcial: componentes funcionales con JSX, los hooks useState y useEffect de React, consumo de API REST utilizando la librería Axios con async/await y manejo de errores mediante try/catch, renderizado dinámico de listas con el método .map(), comunicación entre componentes a través de props, y una arquitectura modular organizada en capas claramente separadas.

El diseño de la interfaz esta construido exclusivamente con TailwindCSS e incluye un modo oscuro y un modo claro que el usuario puede alternar en cualquier momento, animaciones de carga con skeleton cards, y un diseño responsivo que se adapta correctamente a dispositivos móviles, tabletas y computadoras de escritorio.

2. Arquitectura del Proyecto

El proyecto sigue una arquitectura en capas que separa claramente las responsabilidades de cada parte del código. Esta separación facilita el mantenimiento, la escalabilidad y la comprensión del sistema.

2.1 Estructura de carpetas

La organizacion del codigo fuente dentro de la carpeta src/ se divide en tres capas principales:

- **src/components/** — Capa de presentacion: contiene todos los componentes reutilizables de la interfaz de usuario, cada uno con una responsabilidad unica y bien definida.
- **src/pages/** — Capa de paginas: contiene el componente Home.jsx que actua como orquestador principal, gestionando el estado global de la vista y coordinando la comunicacion entre componentes.

- **src/services/** — Capa de servicios: contiene el archivo api.js que centraliza toda la logica de comunicacion con la API de OpenRouter mediante Axios.

2.2 Componentes y sus responsabilidades

Cada componente fue disenado con el principio de responsabilidad unica (SRP):

- **Navbar.jsx:** barra de navegacion fija con el nombre del proyecto, un contador de modelos en tiempo real y el boton para alternar entre modo oscuro y claro.
- **SearchBar.jsx:** input controlado para busqueda en tiempo real. No gestiona su propio estado: recibe el valor y la funcion de actualizacion como props del componente padre.
- **ModelCard.jsx:** tarjeta individual que muestra la informacion de un modelo de IA. Recibe un objeto modelo como prop y lo renderiza con su color distintivo por proveedor.
- **ModelGrid.jsx:** contenedor de la grilla responsiva. Implementa el renderizado dinamico con .map() para crear una ModelCard por cada elemento del array filtrado.
- **Loader.jsx:** spinner de carga con tres anillos orbitales y seis skeleton cards animadas para evitar el layout shift durante la carga inicial.
- **ErrorMessage.jsx:** pantalla de error con mensaje dinamico y boton de reintento. Recibe el mensaje de error y la funcion de reintento como props.

3. Tecnologias Utilizadas

Tecnologia	Version	Rol en el proyecto
React 18	18.x	Liberia principal para construir la interfaz de usuario mediante componentes funcionales reutilizables con JSX.
Vite	6.x	Bundler de nueva generaci3n que ofrece inicio en caliente (HMR) y tiempos de compilaci3n significativamente mas r1pidos que Webpack.
Axios	1.x	Cliente HTTP para el consumo de la API REST. Elegido sobre fetch por su parseo autom1tico de JSON, manejo autom1tico de errores HTTP y soporte de instancias con configuraci3n base.
TailwindCSS	4.x	Framework CSS de utilidades que permite construir interfaces modernas y responsive directamente en el JSX, sin archivos CSS externos adicionales.
JavaScript JSX	ES2024	Lenguaje principal del proyecto. JSX permite escribir la estructura de los componentes de forma declarativa dentro del mismo archivo JavaScript.
OpenRouter API	v1	API REST publica que agrega mas de 300 modelos de IA de distintos proveedores. Endpoint utilizado: GET /api/v1/models.

3.1 Justificación de Axios sobre fetch

La elección de Axios como cliente HTTP en lugar de la API fetch nativa del navegador responde a tres ventajas técnicas concretas:

- Transformación automática de JSON: Axios parsea automáticamente la respuesta como objeto JavaScript. Con fetch, es necesario llamar a `response.json()` en un paso adicional.
- Manejo automático de errores HTTP: Axios lanza excepciones automáticamente cuando el servidor responde con códigos de estado 4xx o 5xx. Fetch considera esas respuestas como exitosas y requiere verificación manual del campo `response.ok`.
- Instancias con configuración base: `axios.create()` permite definir una URL base, headers y timeout comunes para todas las peticiones, evitando la repetición de código y centralizando la configuración en un único lugar.

4. Conclusión

El desarrollo de AI Models Explorer represento un ejercicio integral de aplicacion de los fundamentos del desarrollo de aplicaciones web modernas con React. A lo largo del proyecto se implementaron de manera sistematica y documentada todos los requerimientos academicos estipulados en la rubrica del parcial.

El uso de `useState` permitio gestionar el ciclo de vida completo de los datos en la aplicacion: desde el estado de carga inicial, pasando por el manejo de errores, hasta el estado final con los datos renderizados y filtrados. Cada estado tiene una responsabilidad unica y bien delimitada, lo que resulta en un codigo limpio, predecible y facil de depurar.

La implementacion de `useEffect` con el array de dependencias vacio demostro comprender el ciclo de vida de los componentes de React y la razon fundamental por la que los efectos secundarios, como las llamadas a una API, deben ejecutarse fuera del flujo de renderizado principal para evitar bucles infinitos.

La arquitectura en capas adoptada, con una separacion clara entre la capa de servicios, la capa de paginas y la capa de componentes, refleja las buenas practicas de la industria del desarrollo frontend. Esta separacion garantiza que un cambio en la API solo afecte al archivo `api.js`, sin impactar en los componentes de interfaz de usuario.

La eleccion de Axios como cliente HTTP se fundamenta en ventajas tecnicas objetivas y no en una preferencia arbitraria: el parseo automatico de JSON, el manejo nativo de errores HTTP y la capacidad de crear instancias con configuracion compartida representan mejoras reales en la robustez y mantenibilidad del codigo.

Finalmente, el diseno moderno y responsive construido con TailwindCSS, que incluye modo oscuro, animaciones de carga con skeleton cards, y una interfaz inspirada en las mejores aplicaciones web actuales, demuestra que la calidad tecnica y la calidad visual no son objetivos excluyentes sino complementarios.